fn make_np_sum

Michael Shoemate

October 19, 2026

This proof relies on an idealized model of numeric computation that ignores finite-machine limitations such as non-closure and overflow, so the corresponding API belongs behind the **"idealized-numerics"** feature flag.

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `make_np_sum` in `__init__.py` at commit `3d9816b79` (outdated[1]).

`make_np_sum` accepts an `input_domain` and `input_metric`, and returns a stable Transformation that computes the vector-valued sum with bounded L_p sensitivity.

1 Hoare Triple

Preconditions

None

Pseudocode

```
1
2  def make_np_sum(input_domain: Domain, input_metric: Metric) -> Transformation:
3      """Construct a Transformation that computes a sum over the row axis of a 2-dimensional
       array.
4
5      :param input_domain: instance of `array2_domain(size=_, num_columns=_)`
6      :param input_metric: instance of `symmetric_distance()`
7
8      :return: a Measurement that computes the DP sum
9      """
10     import opendp.prelude as dp
11
12     np = import_optional_dependency("numpy")
13
14     dp.assert_features("contrib", "idealized-numerics")
15
16     if not str(input_domain).startswith("NPArray2Domain"):  #
17         raise ValueError(
18             f"input_domain ({input_domain}) must be NPArray2Domain"
19         )  # pragma: no cover
20
21     if input_domain.descriptor.nan:
22         raise ValueError(
```

[1] See new changes with `git diff 3d9816b79..6b684182 python/src/opendp/extras/numpy/_make_np_sum/__init__.py`

```
            f"input_domain ({input_domain}) must not permit NaN elements"
        )  # pragma: no cover

    if input_metric != dp.symmetric_distance():  #
        raise ValueError(
            "input_metric must be the symmetric distance"
        )  # pragma: no cover

    input_desc = input_domain.descriptor
    norm = input_desc.norm
    if norm is None:  #
        raise ValueError(
            f"input_domain ({input_domain}) must have bounds. See make_np_clamp"
        )  # pragma: no cover

    output_metric = {1: dp.l1_distance, 2: dp.l2_distance}[
        input_desc.p
    ]  #

    if input_desc.size is None:
        origin = np.atleast_1d(input_desc.origin)
        norm += np.linalg.norm(origin, ord=input_desc.p)
        stability = lambda d_in: d_in * norm
    else:
```

Postcondition

Theorem 1.1. For every setting of the input parameters `(input_domain, input_metric)` to `make_np_sum` such that the given preconditions hold,
`make_np_sum` raises an error (at compile time or run time) or returns a valid transformation. A valid transformation has the following properties:

1. (Data-independent runtime errors). For every pair of members x and x' in `input_domain`, `invoke`(x) and `invoke`(x') either both return the same error or neither return an error.

2. (Appropriate output domain). For every member x in `input_domain`, `function`(x) is in `output_domain` or raises a data-independent runtime error.

3. (Stability guarantee). For every pair of members x and x' in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for
`output_metric`, if x, x' are `d_in`-close under `input_metric`, `stability_map(d_in)` does not raise an error, and `stability_map(d_in)` $=$ `d_out`, then `function`(x), `function`(x') are `d_out`-close under `output_metric`.

Proof. Let x and x' be arbitrary members of `input_domain`. By 16, `input_domain` is an `NPArray2Domain`, and by 26, `input_metric` is `SymmetricDistance`. Let R denote `input_domain.descriptor.norm`, and let O denote `input_domain.descriptor.origin`. By 33, R is defined, and by 40, the output metric is the L^p metric with $p \in \{1, 2\}$. Because this proof is under `idealized-numerics`, we ignore machine arithmetic issues such as overflow and treat vector addition as associative and commutative.

Data-independent runtime errors. All runtime checks in `make_np_sum` occur before the transformation is returned. For any member of `input_domain`, the function on line **??** applies `sum(axis=0)` to a 2-dimensional numpy array, which is defined for every such input. Therefore, for every pair $x, x' \in$ `input_domain`, neither `invoke`(x) nor `invoke`(x') raises a runtime error.

Appropriate output domain. Each member of `input_domain` is a 2-dimensional numpy array whose entries have type `input_domain.descriptor.T` and do not contain NaNs. Applying `sum(axis=0)` removes axis zero and returns a 1-dimensional numpy array of the same atomic type. Under idealized numerics, summing non-NaN values yields non-NaN values. Hence `function`(x) is a member of `output_domain`.

Stability guarantee. Fix any `d_in` and `d_out` such that x and x' are `d_in`-close under `input_metric` and `stability_map(d_in)` = `d_out`. Let M and M' be the multisets of rows in x and x'. Let $C = M \cap M'$, $A = M \setminus C$, and $B = M' \setminus C$. Since addition is commutative, all rows in C cancel, so

$$\texttt{function}(x) - \texttt{function}(x') = \sum_{a \in A} a - \sum_{b \in B} b.$$

By the definition of `SymmetricDistance`,

$$|A| + |B| = d_{Sym}(x, x') \leq \texttt{d_in}.$$

Now consider two cases.

Case 1: Known size. If `input_domain.descriptor.size` is known, then x and x' have the same number of rows, so $|A| = |B|$. Let $m = |A| = |B|$. Then $2m \leq$ `d_in`, so $m \leq \lfloor \texttt{d_in}/2 \rfloor$. Enumerate $A = \{a_1, \ldots, a_m\}$ and $B = \{b_1, \ldots, b_m\}$. Since every row lies in the L^p ball of radius R centered at O,

$$\begin{aligned}
\|\texttt{function}(x) - \texttt{function}(x')\|_p &= \left\| \sum_{i=1}^{m} (a_i - b_i) \right\|_p \\
&\leq \sum_{i=1}^{m} \|a_i - b_i\|_p && \text{by triangle inequality} \\
&\leq \sum_{i=1}^{m} \left(\|a_i - O\|_p + \|b_i - O\|_p \right) \\
&\leq \sum_{i=1}^{m} 2R \\
&= 2mR \\
&\leq 2\lfloor \texttt{d_in}/2 \rfloor R.
\end{aligned}$$

This is exactly the known-size stability map returned by `make_np_sum`.

Case 2: Unknown size. If `input_domain.descriptor.size` is unknown, then `make_np_sum` sets

$$\texttt{stability_map}(\texttt{d_in}) = \texttt{d_in} \cdot (R + \|O\|_p).$$

For every row v in either A or B,

$$\|v\|_p \leq \|v - O\|_p + \|O\|_p \leq R + \|O\|_p.$$

Therefore,

$$\begin{aligned}
\|\texttt{function}(x) - \texttt{function}(x')\|_p &= \left\| \sum_{a \in A} a - \sum_{b \in B} b \right\|_p \\
&\leq \sum_{a \in A} \|a\|_p + \sum_{b \in B} \|b\|_p && \text{by triangle inequality} \\
&\leq (|A| + |B|)(R + \|O\|_p) \\
&\leq \texttt{d_in}(R + \|O\|_p).
\end{aligned}$$

This is exactly the unknown-size stability map returned by `make_np_sum`.

In both cases, `function`(x) and `function`(x') are `d_out`-close under `output_metric`. □